

Git Version Control & Repository Management Project | Hands-on Git CLI Practice | DevOps Learning Journey

Project Overview

This hands-on project was performed to understand the core concepts of Git, the most widely used Version Control System in software development and DevOps. The project covers repository creation, file tracking, staging, committing changes, file modification tracking, renaming files, and managing source code history using Git commands.

The practical exercises were executed using Windows PowerShell and Git CLI, providing real-world experience with source code management workflows used by development teams.

◆ Step 1: Creating a Git Repository

Command

```
mkdir raviraj
```

```
cd raviraj
```

```
git init
```

Purpose

The `git init` command initializes a new Git repository inside the project folder and creates a hidden `.git` directory that stores all version control information.

Output

Initialized empty Git repository

Learning Outcome

- Created a local repository.
- Understood how Git starts tracking project history.
- Learned about the hidden `.git` folder.

 Screenshot 1: Git Repository Initialization

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ravi>git --version
git version 2.45.1.windows.1

C:\Users\Ravi>git config --global
usage: git config [<options>]

Config file location
--[no-]global          use global config file
--[no-]system          use system config file
--[no-]local           use repository config file
--[no-]worktree        use per-worktree config file
-f, --[no-]file <file> use given config file
--[no-]blob <blob-id>  read config from given blob object

Action
--[no-]get             get value: name [value-pattern]
--[no-]get-all        get all values: key [value-pattern]
--[no-]get-regexp      get values for regexp: name-regex [value-pattern]
--[no-]get-urlmatch    get value specific for the URL: section[.var] URL
--[no-]replace-all    replace all matching variables: name value [value-pattern]
--[no-]add             add a new variable: name value
--[no-]unset           remove a variable: name [value-pattern]
--[no-]unset-all      remove all matches: name [value-pattern]
--[no-]rename-section  rename section: old-name new-name
--[no-]remove-section  remove a section: name
-l, --[no-]list        list all
```

```
Windows PowerShell
--z, --[no-]null          terminate values with NUL byte
--[no-]name-only         show variable names only
--[no-]includes          respect include directives on lookup
--[no-]show-origins      show origin of config (file, standard input, blob, command line)
--[no-]show-scope        show scope of config (worktree, local, global, system, command)
--[no-]default <value>  with --get, use default value when missing entry
--[no-]comment <value>  human-readable comment string (# will be prepended as needed)

PS C:\Users\Ravi> mkdir raviraj

Directory: C:\Users\Ravi

Mode                LastWriteTime         Length Name
----                -
d-----            30-05-2026     13:10         raviraj

PS C:\Users\Ravi> cd raviraj
PS C:\Users\Ravi\raviraj> git init
Initialized empty Git repository in C:/Users/Ravi/raviraj/.git/
PS C:\Users\Ravi\raviraj> ls
PS C:\Users\Ravi\raviraj> ls -a
```

◆ Step 2: Viewing Hidden Git Files

Command

ls -Force

Purpose

Used to display hidden files and directories, including the .git folder created by Git.

Output

.git

Learning Outcome

- Verified repository initialization.

- Learned that Git stores metadata in the .git folder.

🖼️ Screenshot 2: Viewing Hidden .git Folder

```

Windows PowerShell
PS C:\Users\Ravi\raviraj> ls -Force

Directory: C:\Users\Ravi\raviraj

Mode                LastWriteTime         Length Name
----                -
d--h--            30-05-2026    13:11         .git

PS C:\Users\Ravi\raviraj> git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
PS C:\Users\Ravi\raviraj> open .git
  
```

◆ Step 3: Creating Project Files

Command

echo hello > file.txt

echo hello > file1.txt

Purpose

Created sample text files to demonstrate Git tracking and version control operations.

Output

file.txt

file1.txt **Learning Outcome**

- Created files for Git tracking.
- Prepared files for staging and committing.

```
Windows PowerShell
PS C:\Users\Ravi\raviraj> echo hello > file.txt
PS C:\Users\Ravi\raviraj> ls

Directory: C:\Users\Ravi\raviraj

Mode                LastWriteTime         Length Name
----                -
-a-----          30-05-2026         13:54         16 file.txt

PS C:\Users\Ravi\raviraj> echo hello > file1.txt
PS C:\Users\Ravi\raviraj> ls

Directory: C:\Users\Ravi\raviraj

Mode                LastWriteTime         Length Name
----                -
-a-----          30-05-2026         13:54         16 file.txt
-a-----          30-05-2026         13:55         16 file1.txt

PS C:\Users\Ravi\raviraj> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file.txt
        file1.txt

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Ravi\raviraj> git add .
PS C:\Users\Ravi\raviraj> ls
```

◆ Step 4: Checking Repository Status

Command

git status

Purpose

Displays the current state of the repository, including tracked, modified, and untracked files.

Output

Untracked files:

file.txt

file1.txt

Learning Outcome

- Understood the concept of untracked files.
- Learned how Git identifies new files.

 Screenshot 4: Git Status (Untracked Files)

◆ Step 5: Adding Files to Staging Area

Command

git add .

Purpose

Moves all project files from the Working Directory to the Staging Area.

Output

new file: file.txt

new file: file1.txt

Learning Outcome

- Learned the role of the Staging Area.
- Understood how Git prepares files for commit.

 Screenshot 5: Staging Files

```
Windows PowerShell
PS C:\Users\Ravi\raviraj> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt
        new file:   file1.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

PS C:\Users\Ravi\raviraj> git add . txt
fatal: pathspec 'txt' did not match any files
PS C:\Users\Ravi\raviraj> git add . file.txt
PS C:\Users\Ravi\raviraj> git add . txt
fatal: pathspec 'txt' did not match any files
PS C:\Users\Ravi\raviraj> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt
        new file:   file1.txt

PS C:\Users\Ravi\raviraj> git commit
hint: Waiting for your editor to close the file... |
```

◆ Step 6: Configuring Git User Details

Command

```
git config --global user.name "Ravi Shete"
```

```
git config --global user.email "ravishete261999@gmail.com"
```

Purpose

Sets the Git username and email associated with future commits.

Learning Outcome

- Configured Git identity.
- Learned commit attribution.

 Screenshot 6: Git Configuration

```
C:\Users\Ravi>git config --global user.name "raviraj"
C:\Users\Ravi>git config --global user.email ravishete261999@gmail.com
C:\Users\Ravi>|
```

◆ Step 7: Creating the First Commit

Command

```
git commit -m "Initial Commit"
```

Purpose

Creates a snapshot of all staged files and stores them permanently in Git history.

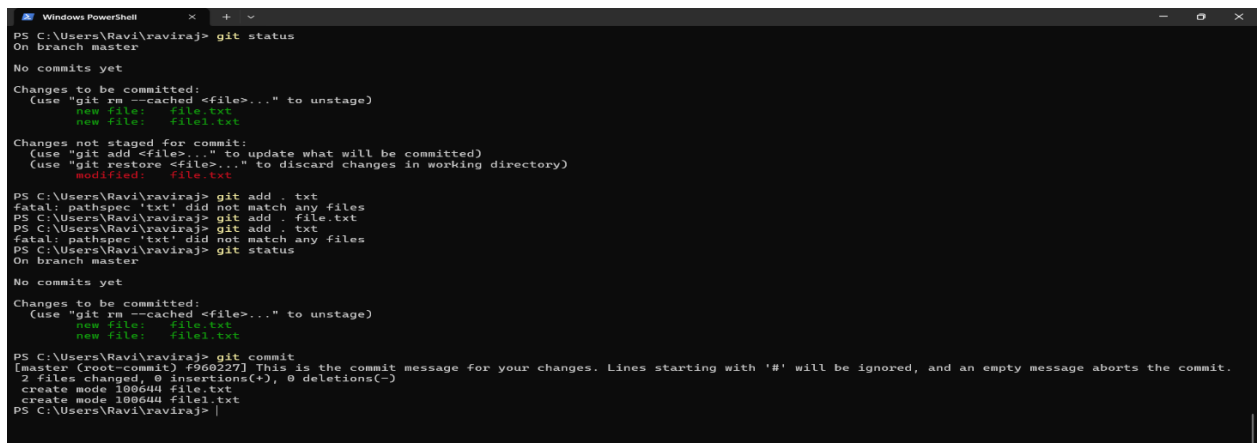
Output

```
[master] Initial Commit
```

Learning Outcome

- Learned how Git stores project versions.
- Understood the importance of commit messages.

📸 Screenshot 7: First Commit



```
Windows PowerShell
PS C:\Users\Ravi\raviraj> git status
On branch master
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt
        new file:   file1.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

PS C:\Users\Ravi\raviraj> git add . txt
fatal: pathspec 'txt' did not match any files
PS C:\Users\Ravi\raviraj> git add . file.txt
PS C:\Users\Ravi\raviraj> git add . txt
fatal: pathspec 'txt' did not match any files
PS C:\Users\Ravi\raviraj> git status
On branch master
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt
        new file:   file1.txt

PS C:\Users\Ravi\raviraj> git commit
[master (root-commit) f960227] This is the commit message for your changes. Lines starting with '#' will be ignored, and an empty message aborts the commit.
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file.txt
 create mode 100644 file1.txt
PS C:\Users\Ravi\raviraj>
```

◆ Step 8: Modifying Existing Files

Command

```
echo world >> file.txt
```

Purpose

Modified an already tracked file to observe how Git detects changes.

Output

```
modified: file.txt
```

Learning Outcome

- Learned change tracking.
- Understood file modification detection.

📸 Screenshot 8: Modified File Detection

```
Windows PowerShell
PS C:\Users\Ravi\raviraj> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt
        new file:   file1.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

PS C:\Users\Ravi\raviraj> git add . txt
fatal: pathspec 'txt' did not match any files
PS C:\Users\Ravi\raviraj> git add . file.txt
PS C:\Users\Ravi\raviraj> git add . txt
fatal: pathspec 'txt' did not match any files
PS C:\Users\Ravi\raviraj> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt
        new file:   file1.txt

PS C:\Users\Ravi\raviraj> git commit
hint: Waiting for your editor to close the file... |
```

◆ Step 9: Updating Repository with New Changes

Command

```
git commit -am "Updated file content"
```

Purpose

Stages and commits changes made to already tracked files.

Learning Outcome

- Learned quick commit workflow.
- Understood difference between tracked and untracked files.

🖼️ Screenshot 9: Updating Existing Files

◆ Step 10: Creating and Tracking New Files

Command

```
echo test > file2.js
```

```
git add file2.js
```

Purpose

Created and tracked a new JavaScript file.

Learning Outcome

- Learned how Git handles newly added files.
- Practiced staging specific files.

 Screenshot 10: Adding New File

◆ Step 11: Renaming Files

Command

```
git mv file.txt main.js
```

Purpose

Renamed an existing tracked file while preserving Git history.

Output

renamed: file.txt -> main.js

Learning Outcome

- Learned Git rename tracking.
- Understood file history preservation.

 Screenshot 11: File Rename Operation

```
PS C:\Users\Ravi\raviraj> mv file.txt main.js
PS C:\Users\Ravi\raviraj> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   file2.txt

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    file.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    main.js

PS C:\Users\Ravi\raviraj> git add file.txt
PS C:\Users\Ravi\raviraj> git add main.js
PS C:\Users\Ravi\raviraj> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   file2.txt
    renamed:    file.txt -> main.js

PS C:\Users\Ravi\raviraj> |
```



```
PS C:\Users\Ravi\raviraj> git mv main.js file1.js
PS C:\Users\Ravi\raviraj> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    file.txt -> file1.js
        new file:   file2.txt

PS C:\Users\Ravi\raviraj> |
```

◆ Step 12: Listing Tracked Files

Command

git ls-files

Purpose

Displays all files currently tracked by Git.

Learning Outcome

- Verified tracked files.
- Understood repository indexing.

🖼️ Screenshot 12: Listing Tracked Files

```
fatal: pathspec 'ls-files' did not match
PS C:\Users\Ravi\raviraj> git ls-files
file.txt
file1.txt
file2.txt
PS C:\Users\Ravi\raviraj> |
```

◆ Step 13: Understanding Git Status Short Format

Command

git status -s

Output Example

M file.txt

A file2.js

?? test.js

Meaning

Symbol Description

M Modified File

A Added File

?? Untracked File

Learning Outcome

- Learned quick repository monitoring.
- Improved Git troubleshooting skills.

 Screenshot 13: Short Status View

```
PS C:\Users\Ravi\raviraj> git echo ocen >> file2.js
git: 'echo' is not a git command. See 'git --help'.

The most similar command is
    fetch
PS C:\Users\Ravi\raviraj> git status -s
 M file2.txt
?? file2.js
PS C:\Users\Ravi\raviraj> git add file.js
fatal: pathspec 'file.js' did not match any files
PS C:\Users\Ravi\raviraj> git add file2.js
PS C:\Users\Ravi\raviraj> git status -s
A  file2.js
 M file2.txt
PS C:\Users\Ravi\raviraj> |
```

◆ Step 14: Viewing Commit History

Command

git log

Purpose

Displays the complete commit history of the repository, including commit IDs, author information, date, and commit messages.

Output Example

commit 1f869e3...

Author: Ravi Shete

Date: Sat May 30

add

commit c4a1b96...

Author: Ravi Shete

Date: Sat May 30

remove the bin dir

Learning Outcome

- Learned how Git stores project history.
- Understood commit tracking.
- Learned how to identify previous versions of the project.

 Screenshot 14: Git Log History

```
Windows PowerShell
Viewing (1/1): 'file2.js'
Launch 'vscode' [Y/n]? y
PS C:\Users\Ravi\raviraj> git log
WARNING: terminal is not fully functional
Press RETURN to continue
commit 1f869e31547cf3f71ff60160b837902d24b58aee (HEAD -> master)
Author: raviraj <ravishete261999@gmail.com>
Date: Sat May 30 14:43:32 2026 +0530

    add

commit c4a1b96c2f5b0bfe4a054802e623c639aa0d9fc4
Author: raviraj <ravishete261999@gmail.com>
Date: Sat May 30 14:39:27 2026 +0530

    remove the bin dir

commit c86c3392c30d278c759631a2d562c4500d85a0eb
Author: raviraj <ravishete261999@gmail.com>
Date: Sat May 30 14:33:21 2026 +0530

    add bin.

commit 1b89efa83dd0d706b6768d4e5023d7321fd2786f
Author: raviraj <ravishete261999@gmail.com>
Date: Sat May 30 14:31:09 2026 +0530

    add gitignore

commit 414931e84f266f247fdef3f18ce5d12c1af0516e
Author: raviraj <ravishete261999@gmail.com>
Date: Sat May 30 14:24:13 2026 +0530

    code refactor

commit 53245579bb9992101251ddfb0fe3f5f176f9c07f
Author: raviraj <ravishete261999@gmail.com>
Date: Sat May 30 14:11:53 2026 +0530

    all bug done
```

1 38°C
Sunny

Search

◆ Step 15: Viewing Commit History in One Line

Command

`git log --oneline`

Purpose

Displays a compact version of commit history.

Output Example

cbf6d2d delete

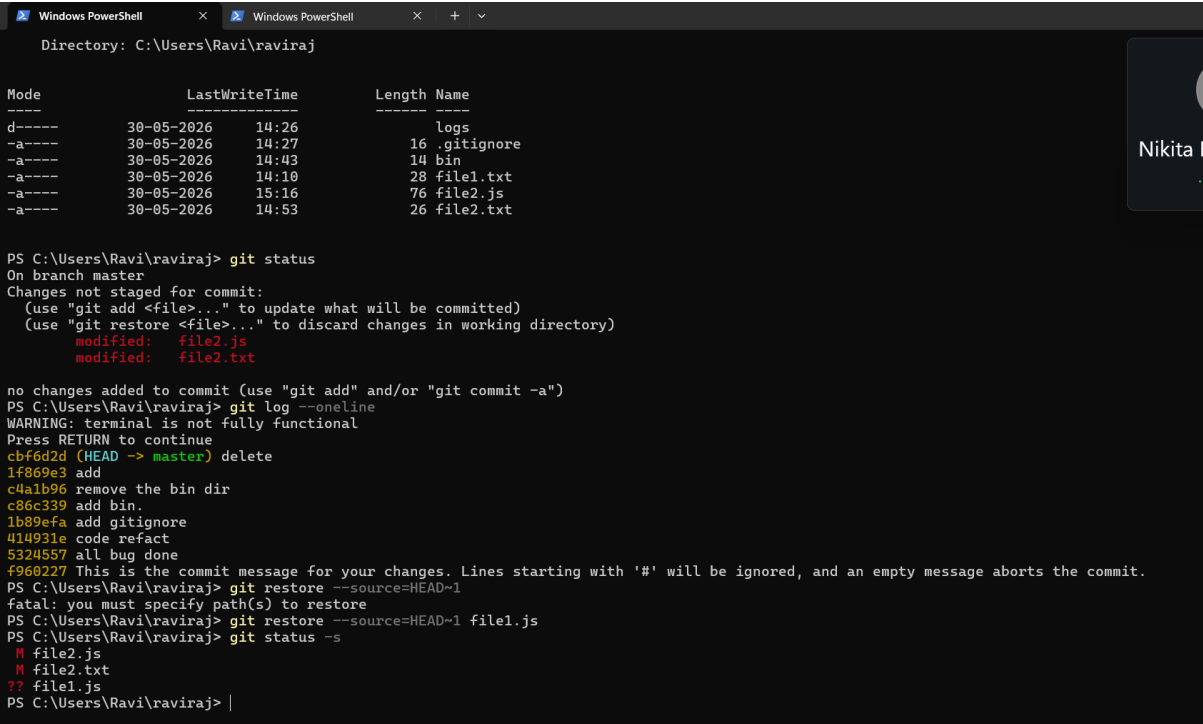
1f869e3 add

c4a1b96 remove the bin dir

Learning Outcome

- Quickly review commit history.
- Easily identify commit IDs.
- Useful for rollback and restore operations.

Screenshot 15: Git Log One-Line View



```
Windows PowerShell
Directory: C:\Users\Ravi\raviraj

Mode                LastWriteTime         Length Name
----                -
d-----          30-05-2026   14:26             logs
-a-----          30-05-2026   14:27             16 .gitignore
-a-----          30-05-2026   14:43             14 bin
-a-----          30-05-2026   14:10            28 file1.txt
-a-----          30-05-2026   15:16            76 file2.js
-a-----          30-05-2026   14:53            26 file2.txt

PS C:\Users\Ravi\raviraj> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file2.js
        modified:   file2.txt

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Ravi\raviraj> git log --oneline
WARNING: terminal is not fully functional
Press RETURN to continue
cbf6d2d (HEAD -> master) delete
1f869e3 add
c4a1b96 remove the bin dir
c86c339 add bin.
1b89efa add gitignore
414931e code refactor
5324557 all bug done
f960227 This is the commit message for your changes. Lines starting with '#' will be ignored, and an empty message aborts the commit.
PS C:\Users\Ravi\raviraj> git restore --source=HEAD~1
fatal: you must specify path(s) to restore
PS C:\Users\Ravi\raviraj> git restore --source=HEAD~1 file1.js
PS C:\Users\Ravi\raviraj> git status -s
 M file2.js
 M file2.txt
?? file1.js
PS C:\Users\Ravi\raviraj> |
```

◆ Step 16: Viewing Previous Commits

Command

`git show HEAD~1`

`git show HEAD~2`

Purpose

Displays detailed information about previous commits, including file changes and differences.

Output

```
commit c4a1b96...
```

```
remove the bin dir
```

Learning Outcome

- Learned how to inspect historical commits.
- Understood Git's version tracking mechanism.
- Viewed differences between repository states.

 Screenshot 16: Git Show Previous Commits

◆ Step 17: Viewing Files from Previous Commits

Command

```
git ls-tree HEAD~1
```

Purpose

Lists all files and directories that existed in a previous commit.

Output Example

```
.gitignore
```

```
file1.js
```

```
file1.txt
```

```
file2.txt
```

```
logs
```

Learning Outcome

- Learned how to inspect repository contents from older commits.
- Understood how Git stores snapshots.

 Screenshot 17: Git LS-Tree

◆ Step 18: Soft Reset

Command

```
git reset --soft HEAD~1
```

Purpose

Moves the HEAD pointer back by one commit while keeping all changes staged.

Learning Outcome

- Learned how to undo commits safely.
- Understood Git history manipulation.
- Preserved changes while removing commit history.

 Screenshot 18: Git Reset Soft

◆ **Step 19: Removing Files from Repository**

Command

```
git rm file1.js
```

Purpose

Removes a tracked file from both the working directory and Git repository.

Output

```
rm 'file1.js'
```

Learning Outcome

- Learned file deletion tracking.
- Understood how Git handles file removal.

 Screenshot 19: Git Remove File

◆ **Step 20: Committing File Deletion**

Command

```
git commit -m "delete"
```

Purpose

Records file deletion changes into Git history.

Output

```
[master cbf6d2d] delete
```

Learning Outcome

- Learned how Git tracks deleted files.
- Maintained complete project history.

◆ Step 21: Restoring Files from Previous Commit

Command

```
git restore --source=HEAD~1 file1.js
```

Purpose

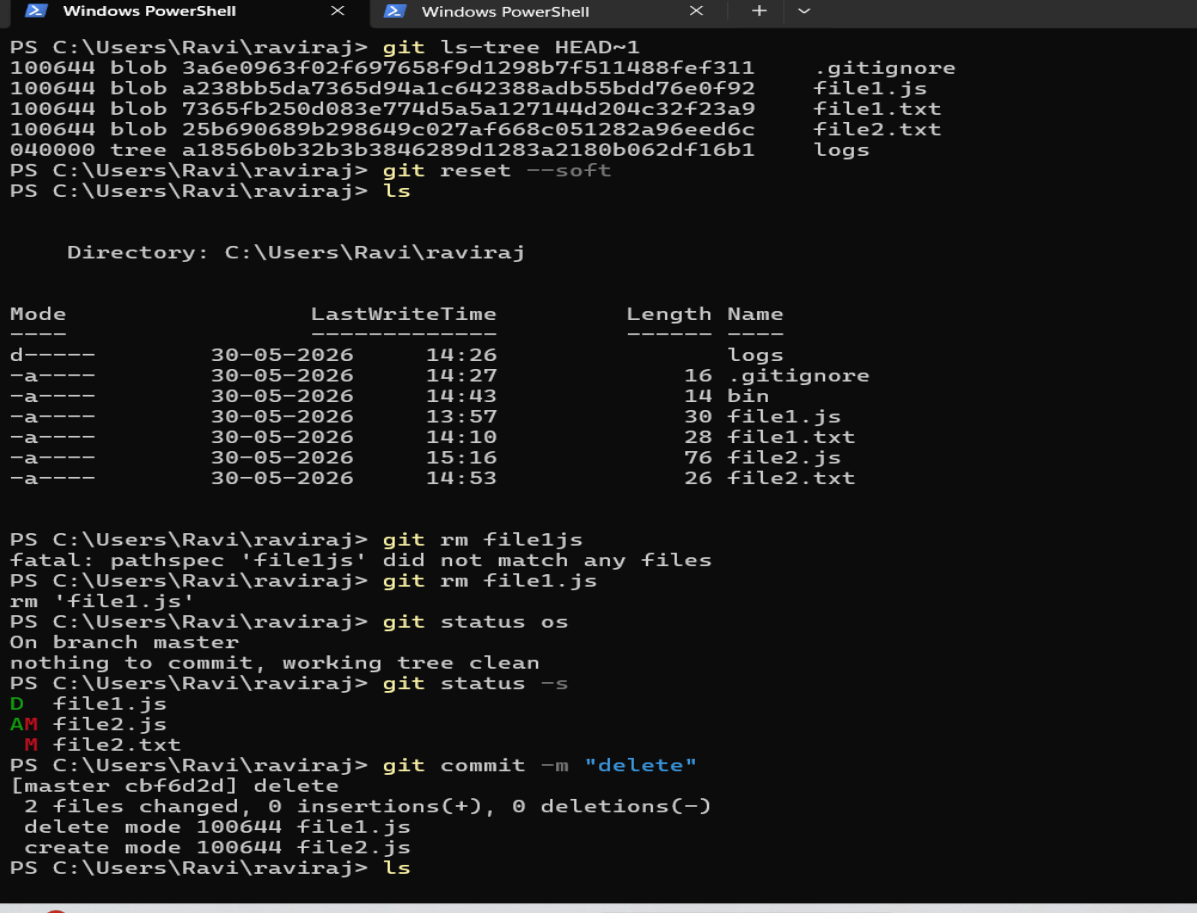
Restores a deleted file from an earlier commit without affecting other project files.

Output

file1.js restored

Learning Outcome

- Learned disaster recovery using Git.
- Restored accidentally deleted files.
- Understood Git's snapshot-based architecture.



```
PS C:\Users\Ravi\raviraj> git ls-tree HEAD~1
100644 blob 3a6e0963f02f697658f9d1298b7f511488fef311 .gitignore
100644 blob a238bb5da7365d94a1c642388adb55bdd76e0f92 file1.js
100644 blob 7365fb250d083e774d5a5a127144d204c32f23a9 file1.txt
100644 blob 25b690689b298649c027af668c051282a96eed6c file2.txt
040000 tree a1856b0b32b3b3846289d1283a2180b062df16b1 logs
PS C:\Users\Ravi\raviraj> git reset --soft
PS C:\Users\Ravi\raviraj> ls

Directory: C:\Users\Ravi\raviraj

Mode                LastWriteTime         Length Name
----                -
d-----          30-05-2026         14:26     logs
-a-----          30-05-2026         14:27         16 .gitignore
-a-----          30-05-2026         14:43         14 bin
-a-----          30-05-2026         13:57         30 file1.js
-a-----          30-05-2026         14:10         28 file1.txt
-a-----          30-05-2026         15:16         76 file2.js
-a-----          30-05-2026         14:53         26 file2.txt

PS C:\Users\Ravi\raviraj> git rm file1js
fatal: pathspec 'file1js' did not match any files
PS C:\Users\Ravi\raviraj> git rm file1.js
rm 'file1.js'
PS C:\Users\Ravi\raviraj> git status os
On branch master
nothing to commit, working tree clean
PS C:\Users\Ravi\raviraj> git status -s
D  file1.js
AM file2.js
M  file2.txt
PS C:\Users\Ravi\raviraj> git commit -m "delete"
[master cbf6d2d] delete
2 files changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 file1.js
create mode 100644 file2.js
PS C:\Users\Ravi\raviraj> ls
```


◆ Step 22: Git Diff Tool Configuration in VS Code

Command

```
git config --global diff.tool vscode
```

Configuration

```
[diff]
```

```
tool = vscode
```

```
[difftool "vscode"]
```

```
cmd = "code --wait --diff $LOCAL $REMOTE"
```

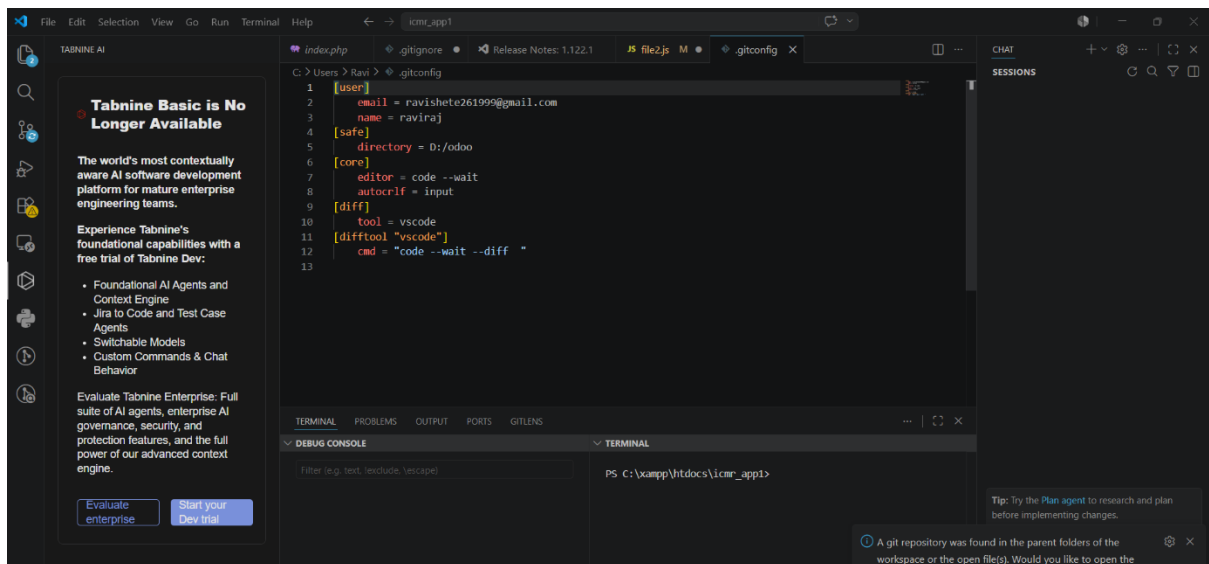
Purpose

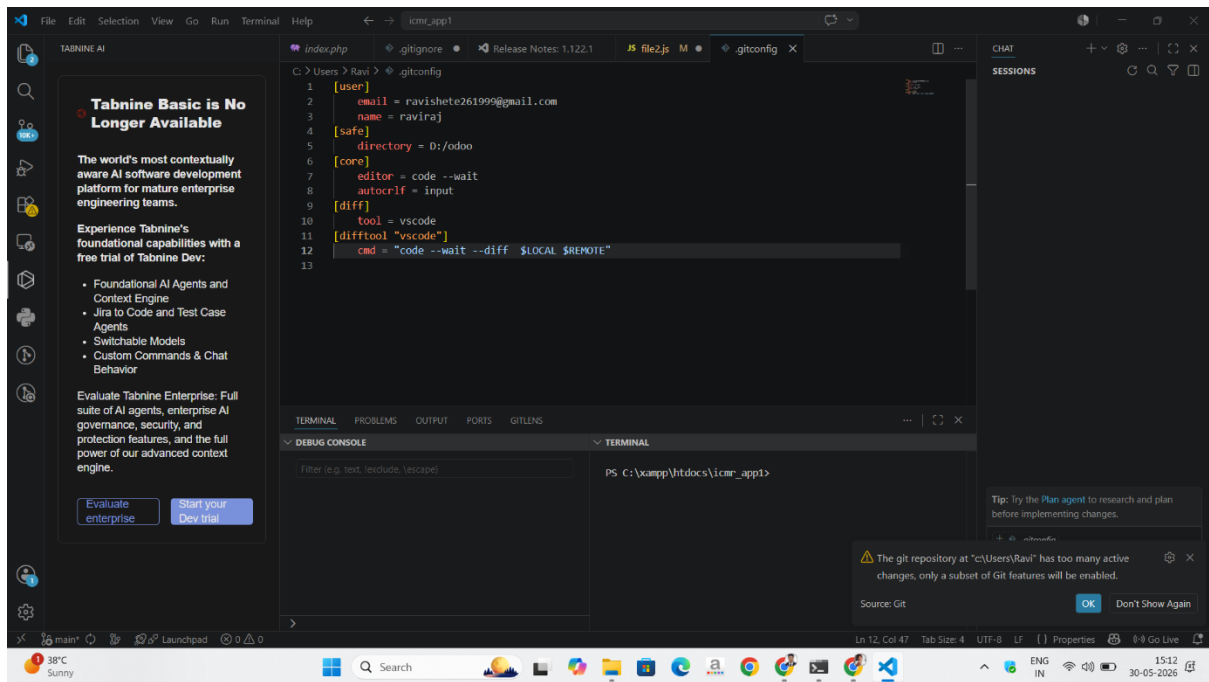
Configured Visual Studio Code as Git's default comparison tool.

Learning Outcome

- Integrated Git with VS Code.
- Learned graphical file comparison.
- Improved code review workflow.

📸 Screenshot 22: VS Code Diff Tool Configuration





Step 23: Visual File Difference Comparison

Purpose

Used VS Code's built-in diff viewer to compare file versions and identify changes line by line.

Features Observed

- ✓ Added Lines (Green)
- ✓ Deleted Lines (Red)
- ✓ Side-by-Side Comparison
- ✓ Change Tracking

Learning Outcome

- Learned visual comparison of code versions.
- Improved debugging and review skills.
- Understood Git diff workflow.

 Screenshot 23: VS Code Diff Viewer

```
Windows PowerShell
c4a1b96 remove the bin dir
1f869e3 (HEAD -> master) add
PS C:\Users\Ravi\raviraj> git head~1
git: 'head~1' is not a git command. See 'git --help'.
PS C:\Users\Ravi\raviraj>
PS C:\Users\Ravi\raviraj> git show HEAD~1
WARNING: terminal is not fully functional
Press RETURN to continue
commit c4a1b96c2f5b0bfe4a054802e623c639aa0d9fc4
Author: raviraj <ravishete261999@gmail.com>
Date: Sat May 30 14:39:27 2026 +0530

remove the bin dir

diff --git a/bin/app.bin b/bin/app.bin
deleted file mode 100644
index 2a93d00..0000000
Binary files a/bin/app.bin and /dev/null differ
PS C:\Users\Ravi\raviraj> git show HEAD~2
WARNING: terminal is not fully functional
Press RETURN to continue
commit c86c3392c30d278c759631a2d562c4500d85a0eb
Author: raviraj <ravishete261999@gmail.com>
Date: Sat May 30 14:33:21 2026 +0530

add bin.

diff --git a/bin/app.bin b/bin/app.bin
new file mode 100644
index 0000000..2a93d00
Binary files /dev/null and b/bin/app.bin differ
PS C:\Users\Ravi\raviraj> git ls-tree
usage: git ls-tree [<options>] <tree-ish> [<path>...]

    -d                only show trees
    -r                recurse into subtrees
    -t                show trees when recursing
    -z                terminate entries with NUL byte
    -l, --long        include object size
    --name-only        list only filenames
    --name-status      list only filenames
```

- git init
- git status
- git status -s
- git add
- git add .
- git commit
- git commit -m
- git commit -am
- git config
- git log
- git log --oneline
- git show
- git ls-files
- git ls-tree
- git mv
- git rm
- git restore
- git reset --soft



Concepts Covered

- ✓ Repository Initialization
 - ✓ Git Configuration
 - ✓ Working Directory Management
 - ✓ Staging Area Operations
 - ✓ Local Repository Management
 - ✓ Commit Creation & History Tracking
 - ✓ Commit Inspection & Review
 - ✓ File Tracking & Version Control
 - ✓ File Modification Detection
 - ✓ File Renaming with History Preservation
 - ✓ File Deletion & Recovery
 - ✓ Git Restore & Version Recovery
 - ✓ Soft Reset Operations
 - ✓ Git Log Analysis
 - ✓ Git Diff & Change Comparison
 - ✓ VS Code Integration with Git
 - ✓ Repository Navigation & File Tree Inspection
-



Skills Gained

Version Control & Source Code Management

- Git Repository Management
- Source Code Tracking
- Change Management
- Version Recovery
- Commit Management
- Repository Navigation

Git Operations

- Repository Initialization
- File Staging & Tracking
- Commit Creation & Management
- File Rename Operations
- File Restore & Recovery
- Commit History Analysis
- Repository Reset Operations

Development Tools

- Git CLI (Command Line Interface)
- Windows PowerShell
- Visual Studio Code Integration
- Git Diff Tool Configuration



Key Learning Outcomes

- Developed hands-on experience with Git Version Control workflows.
- Learned how Git manages source code through Working Directory, Staging Area, and Repository architecture.
- Practiced creating, modifying, deleting, restoring, and tracking files using Git commands.
- Gained practical knowledge of commit history management, rollback strategies, and version recovery techniques.
- Configured Visual Studio Code as a Git Diff Tool for graphical comparison of file changes.
- Strengthened foundational skills required for DevOps, Cloud Engineering, CI/CD Pipelines, and Collaborative Software Development.

Future Enhancements

The next phase of learning will focus on:

- Git Branching Strategy
- Branch Creation & Management
- Git Merge & Conflict Resolution
- Git Rebase
- Git Stash
- Git Tags
- GitHub Integration
- Remote Repository Management
- Clone, Pull, Push Operations
- Pull Requests (PR)
- Code Review Workflow
- GitHub Actions
- CI/CD Pipeline Integration
- Docker & Kubernetes Source Control Workflow